PROJECT REPORT

STUDENT RECORD MANAGEMENT SYSTEM(SRMS)

DONE BY

NAME: Md. Afrid

REGISTRATION NUMBER: AP24110011860

CSE 2nd YEAR, THIRD SEMESTER

SECTION W

SUBMITTED TO

RAKESH RAMA RAJU

CODING SKILLS-1 (CSE 201)

ABSTRACT

The **Student Report Management System (SRMS)** is a console-based application developed in the C programming language to efficiently manage student academic records. The system enables secure login and provides two roles: **Admin and User**. Admins can add, display, search, update, and delete student reports, whereas users can only view and search student data. All records are stored in external text files, ensuring portability and simplicity. This project demonstrates the use of **file handling, structures, conditional statements, and user authentication** in C programming. The system provides an easy and reliable solution for maintaining student academic details.

The **Student Report Management System (SRMS)** is a console-based application developed in the C programming language to efficiently maintain academic information of students. The system enables secure login authentication and supports two separate user roles: **Admin** and **User**. Administrators are provided with complete access to manage student data, including operations such as adding new records, viewing the list of students, searching based on roll number, updating existing details, and deleting incorrect or outdated entries. Meanwhile, general users are restricted to viewing and searching student information only, ensuring controlled privilege management.

The system makes use of data persistence through **file handling operations**, storing user credentials as well as student data in text files without requiring any external database. By utilizing concepts such as **structures, loops, conditional logic, functions, and file handling**, SRMS presents a practical approach to digital record management. It enhances efficiency, reduces manual errors, and provides a user-friendly solution for institutions to maintain academic records securely and systematically.

INTRODUCTION

The Student Report Management System (SRMS) is designed to provide a simple yet structured method for storing and accessing student academic details digitally. Developed using the C programming language, the system provides basic but essential functionalities required for managing student records efficiently. It mainly focuses on storing each student's roll number, name, and academic marks in a secure and manageable format. Instead of relying on complex databases, SRMS uses text file storage to maintain the data permanently, which makes the application lightweight and easily deployable on any computer supporting the C environment.

The system incorporates user authentication, which ensures that only authorized individuals can access or modify student information. Two types of users are allowed to log into the system: Admin and User. The Admin has complete control over the student database, with privileges such as adding new student details, modifying existing data, and deleting inaccurate or outdated records. On the other hand, a regular User is limited to viewing and searching student data only, ensuring that critical information cannot be altered by unauthorized users. This role-based access control enhances data security and minimizes the risks of unauthorized modifications.

One of the key programming aspects of SRMS is the use of structures in C, which allow grouping of student information into a single data type. Along with structures, the project utilizes file handling, allowing data to be written, read, searched, updated, and deleted from text files. It also incorporates modular programming through user-defined functions, which improves readability, reusability, and maintenance of the code. Control statements such as loops, condition checking, and string manipulation functions are used to validate data, compare credentials, and process user input efficiently.

OBJECTIVES

THE KEY OBJECTIVES OF THIS PROJECT ARE:

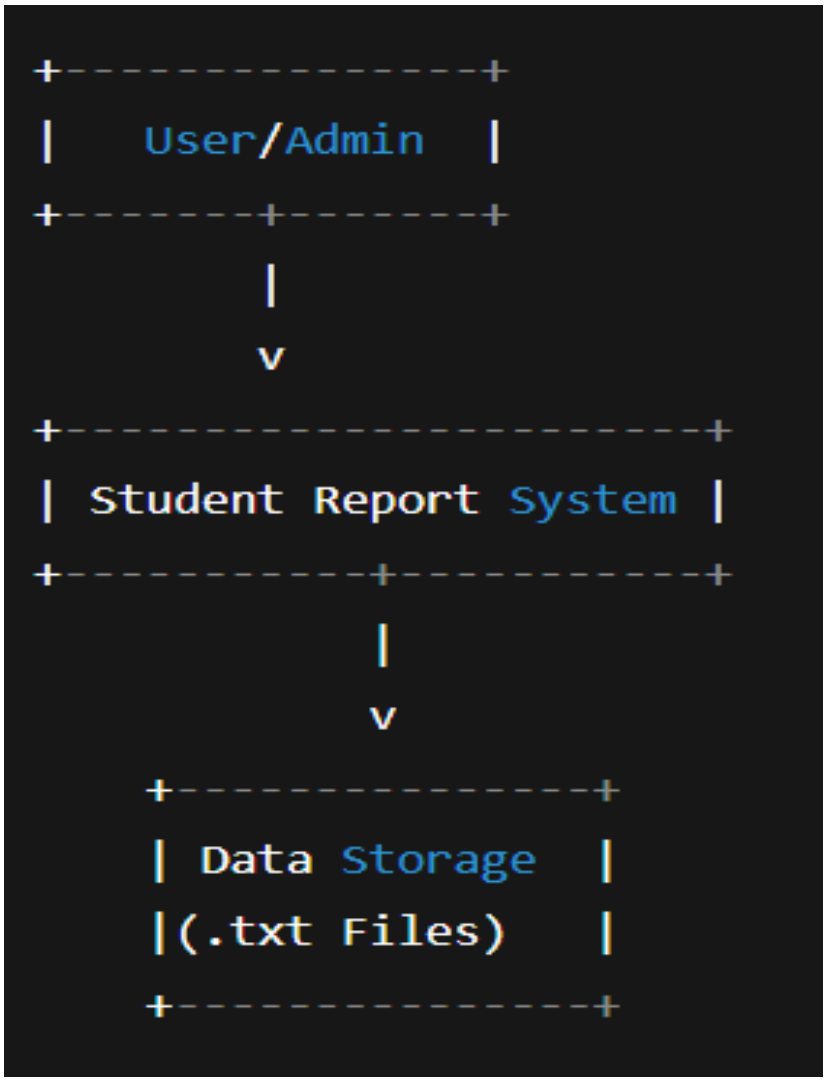
- To create a computerized system that efficiently maintains academic records of students.
- To provide a **secure login facility** to protect stored student information from unauthorized access.
- To implement **role-based access**, ensuring Admin users have full control while general users have limited privileges.
- To maintain student information such as **roll number, name, and marks** in a well-structured format.
- To replace traditional manual record-keeping with a **fast and error-free digital solution**.
- To allow **easy addition of new student data** without overwriting existing records.
- To provide a quick way to **search student records** based on roll number.
- To enable **updating of student details**, ensuring the database always remains accurate.
- To offer an option to **delete outdated or wrong records** efficiently without affecting other data.
- To use **file handling techniques** for storing student data permanently without using external databases.
- To demonstrate the real-world application of **C programming concepts**, including structures and modular programming.

MODULES

Module Name	Description
Login Module	Authenticates user credentials and supports role-based access (Admin/User).
Admin Module	Allows administrators to add, view, search, update, and delete student records.
User Module	Allows viewing and searching of student data only (restricted access).
Add Student Module	Stores new student data (Roll No., Name, Marks).
View Records Module	Displays all students' stored records from the file.
Search Module	Searches for a student using their roll number.
Update/Delete Modules	Admins can modify or remove stored student data.
File Storage Module	Handles credential storage (credentials.txt) and student records (students.txt).

DATA FLOW DIAGRAM

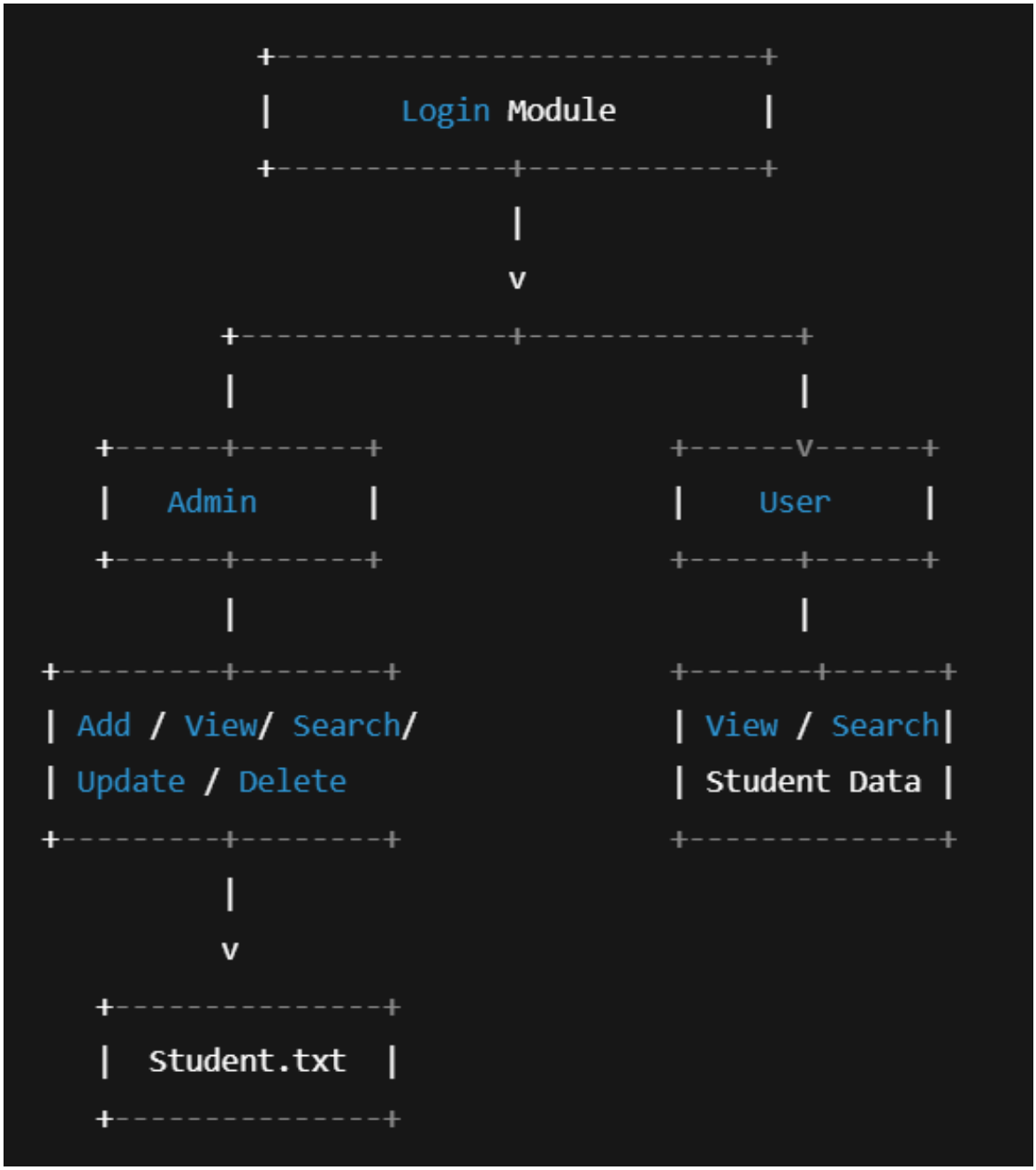
LEVEL -1 DATA FLOW DIAGRAM



Explanation

- User/Admin interacts with the SRMS.
- System processes the request and accesses stored files.

LEVEL -2 DATA FLOW DIAGRAM



Explanation

- After login, control is transferred based on role:
  - Admin: Full access (Add, View, Search, Update, Delete).
  - User: Only View and Search.
- All actions read/write from **students.txt**.

CODE & OUTPUTS

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define STUD_FILE "students.txt"
#define CRE_FILE  "credentials.txt"

char currentUser[50];
char currentRole[20];

int login() {
    char u[50], p[50], r[20];
    char inUser[50], inPass[50];

    printf("USERNAME: ");
    scanf("%s", inUser);
    printf("PASSWORD: ");
    scanf("%s", inPass);

    FILE *fp = fopen(CRE_FILE, "r");
    if (!fp) {
        printf("Credential file missing!\n");
        return 0;
    }

    while (fscanf(fp, "%s %s %s", u, p, r) == 3) {
        if (strcmp(inUser, u) == 0 && strcmp(inPass, p) == 0) {
            strcpy(currentUser, u);
            strcpy(currentRole, r);
            fclose(fp);
            return 1;
        }
    }

    fclose(fp);
    return 0;
}

void addStudent() {
    int roll;
    char name[50];
    float mark;

    printf("Roll: ");
    scanf("%d", &roll);
    printf("Name: ");
    scanf(" %[^\n]", name);
    printf("Mark: ");
    scanf("%f", &mark);

    FILE *fp = fopen(STUD_FILE, "a");
```

```
    fprintf(fp, "%d %s %.2f\n", roll, name, mark);
    fclose(fp);

    printf("Student added!\n");
}

void displayStudents() {
    FILE *fp = fopen(STUD_FILE, "r");
    if (!fp) {
        printf("No student file!\n");
        return;
    }

    int roll;
    char name[50];
    float mark;

    printf("Roll\tName\tMark\n");
    printf("----\t----\t----\n");
    while (fscanf(fp, "%d %s %f", &roll, name, &mark) == 3) {
        printf("%d\t%s\t%.2f\n", roll, name, mark);
    }

    fclose(fp);
}

void searchStudent() {
    int find, roll;
    char name[50];
    float mark;

    printf("Enter roll to search: ");
    scanf("%d", &find);

    FILE *fp = fopen(STUD_FILE, "r");
    while (fscanf(fp, "%d %s %f", &roll, name, &mark) == 3) {
        if (roll == find) {
            printf("Found: %d %s %.2f\n", roll, name, mark);
            fclose(fp);
            return;
        }
    }
    fclose(fp);
    printf("Student not found!\n");
}

void deleteStudent() {
    int delRoll;
    printf("Enter roll to delete: ");
    scanf("%d", &delRoll);

    FILE *fp = fopen(STUD_FILE, "r");
    FILE *temp = fopen("temp.txt", "w");

    int roll;
    char name[50];
    float mark;
    int found = 0;
```

```
    while (fscanf(fp, "%d %s %f", &roll, name, &mark) == 3) {
        if (roll != delRoll) {
            fprintf(temp, "%d %s %.2f\n", roll, name, mark);
        } else {
            found = 1;
        }
    }

    fclose(fp);
    fclose(temp);

    remove(STUD_FILE);
    rename("temp.txt", STUD_FILE);

    if (found) printf("Student deleted!\n");
    else printf("Roll not found!\n");
}

void updateStudent() {
    int updateRoll;
    printf("Enter roll to update: ");
    scanf("%d", &updateRoll);

    FILE *fp = fopen(STUD_FILE, "r");
    FILE *temp = fopen("temp.txt", "w");

    int roll;
    char name[50];
    float mark;
    int found = 0;

    while (fscanf(fp, "%d %s %f", &roll, name, &mark) == 3) {
        if (roll == updateRoll) {
            found = 1;
            char newName[50];
            float newMark;

            printf("New Name: ");
            scanf(" %[^\n]", newName);
            printf("New Mark: ");
            scanf("%f", &newMark);

            fprintf(temp, "%d %s %.2f\n", roll, newName, newMark);
        } else {
            fprintf(temp, "%d %s %.2f\n", roll, name, mark);
        }
    }

    fclose(fp);
    fclose(temp);

    remove(STUD_FILE);
    rename("temp.txt", STUD_FILE);

    if (found) printf("Student updated!\n");
    else printf("Roll not found!\n");
}
```

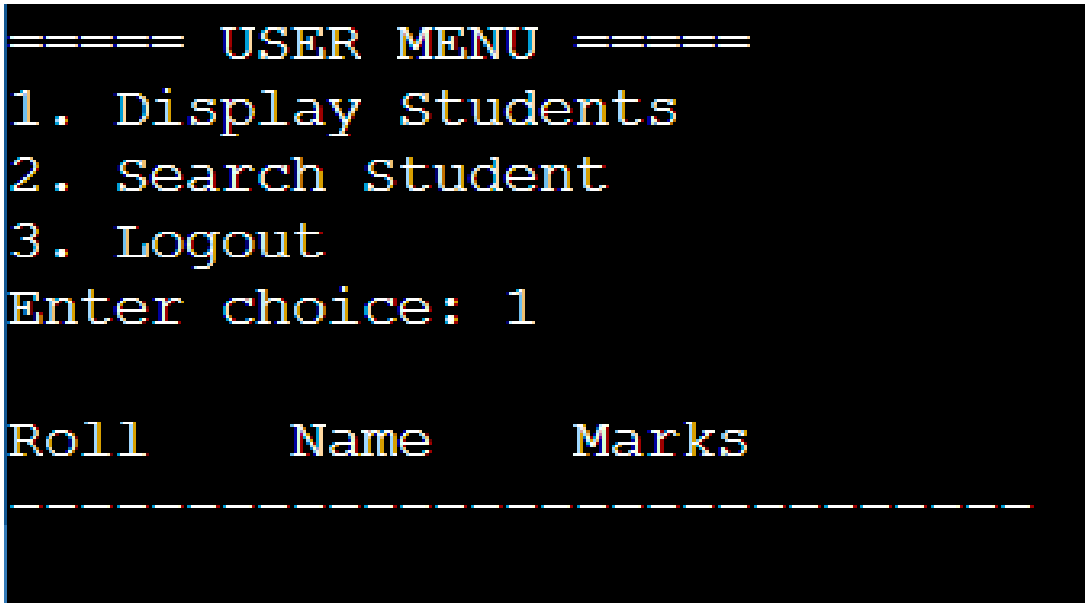
```
void adminMenu() {
    int c;
    while (1) {
        printf("\nADMIN MENU\n");

printf("1.Add\n2.Display\n3.Search\n4.Update\n5.Delete\n6.Logout\n");
        scanf("%d",&c);
        if(c==1)addStudent();
        else if(c==2)displayStudents();
        else if(c==3)searchStudent();
        else if(c==4)updateStudent();
        else if(c==5)deleteStudent();
        else return;
    }
}

void staffMenu() {
    int c;
    while (1) {
        printf("\nSTAFF MENU\n");
        printf("1.Add\n2.Display\n3.Search\n4.Update\n5.Logout\n");
        scanf("%d",&c);
        if(c==1)addStudent();
        else if(c==2)displayStudents();
        else if(c==3)searchStudent();
        else if(c==4)updateStudent();
        else return;
    }
}

void guestMenu() {
    int c;
    while (1) {
        printf("\nGUEST MENU\n");
        printf("1.Display\n2.Search\n3.Logout\n");
        scanf("%d",&c);
        if(c==1)displayStudents();
        else if(c==2)searchStudent();
        else return;
    }
}

int main() {
    if (!login()) {
        printf("Invalid login!\n");
        return 0;
    }

    printf("Logged in as: %s\n", currentRole);

    if (strcmp(currentRole,"admin")==0) adminMenu();
    else if (strcmp(currentRole,"staff")==0) staffMenu();
    else guestMenu();

    return 0;
}
```

OUTPUTS:

```
Do you want to create login credentials? (y/n): y
Enter new username: admin
Enter new password: admin123
Enter role (admin/user): admin

Credentials created successfully!

============== LOGIN SCREEN ============
Enter username: admin
Enter password: admin123
```

```
===== ADMIN MENU =====
1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete Student
6. Logout
Enter choice: 1
Enter roll number: ap24001100976
Enter name: Enter marks: saicharan:560
Student added successfully!
```

```
Do you want to create login credentials? (y/n): y
Enter new username: user
Enter new password: user123
Enter role (admin/user): user

Credentials created successfully!

============== LOGIN SCREEN ============
Enter username: user
Enter password: user123
```



credentials.txt

admin admin123 admin

staff staff123 staff

guest guest123 guest

students.txt

1 John 78.5

2 Maya 88.0

CONCLUSION

The Student Report Management System (SRMS) is an effective and user-friendly solution designed to simplify the management of student academic information. The project successfully demonstrates how fundamental C programming concepts such as structures, file handling, decision making, and modular coding techniques can be applied to develop real-time applications. Through its secure login facility and role-based access control, SRMS ensures proper data privacy by allowing different access rights to administrators, staff, and guest users. All academic records are stored in text files, making the system lightweight, portable, and easy to use without requiring any external database.

The system provides all essential data operations such as adding, updating, deleting, searching, and viewing student details. It significantly reduces the chances of human error compared to manual record management and enhances efficiency by offering quick retrieval of student information. This project highlights the importance of digitizing academic data and serves as a strong foundation for further enhancements, such as integrating databases, graphical interfaces, encryption, and cloud support. Overall, SRMS proves to be an impactful and practical software application that can be adopted by educational institutions of different scales to streamline their record-keeping process.